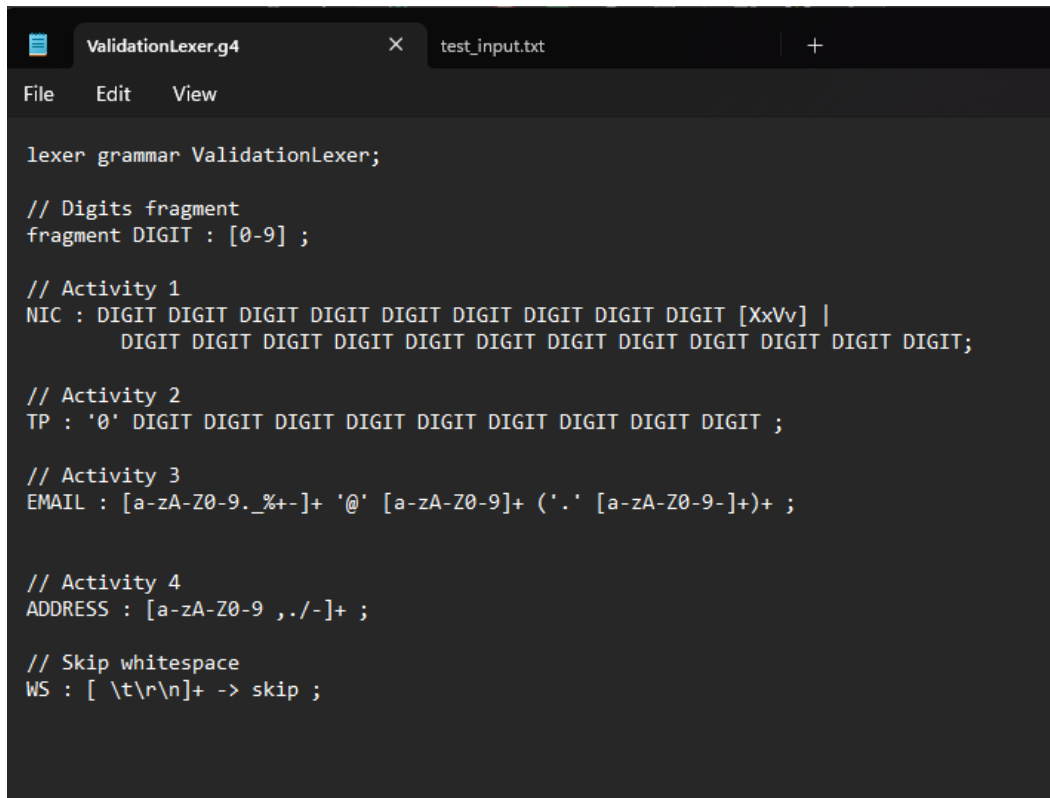


23002192

Tutorial: ANTLR Lexer Design

Activity: 1/2/3/4



```
lexer grammar ValidationLexer;

// Digits fragment
fragment DIGIT : [0-9] ;

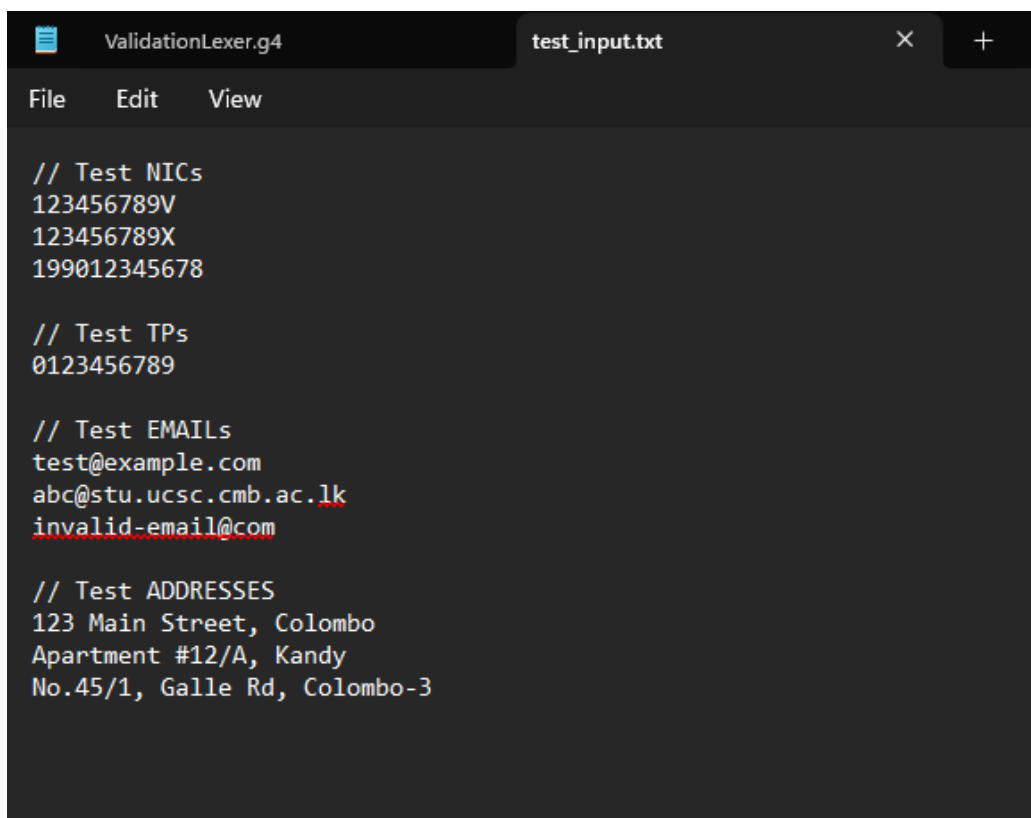
// Activity 1
NIC : DIGIT DIGIT DIGIT DIGIT DIGIT DIGIT DIGIT DIGIT DIGIT [XxVv] |
    DIGIT DIGIT DIGIT DIGIT DIGIT DIGIT DIGIT DIGIT DIGIT DIGIT;

// Activity 2
TP : '0' DIGIT DIGIT DIGIT DIGIT DIGIT DIGIT DIGIT DIGIT DIGIT ;

// Activity 3
EMAIL : [a-zA-Z0-9._%+-]+ '@' [a-zA-Z0-9]+ ('.' [a-zA-Z0-9-]+)+ ;

// Activity 4
ADDRESS : [a-zA-Z0-9 ,./-]+ ;

// Skip whitespace
WS : [ \t\r\n]+ -> skip ;
```



```
// Test NICs
123456789V
123456789X
199012345678

// Test TPs
0123456789

// Test EMAILs
test@example.com
abc@stu.ucsc.cmb.ac.lk
invalid-email@com

// Test ADDRESSES
123 Main Street, Colombo
Apartment #12/A, Kandy
No.45/1, Galle Rd, Colombo-3
```

```

Administrator: Windows PowerShell
PS C:\antlr-projects\ANTLR_Lexer_Tutorial> notepad Main.java
PS C:\antlr-projects\ANTLR_Lexer_Tutorial> code Main.java
PS C:\antlr-projects\ANTLR_Lexer_Tutorial> javac Main.java
Note: Main.java uses or overrides a deprecated API.
Note: Recompile with -Xlint:deprecation for details.
PS C:\antlr-projects\ANTLR_Lexer_Tutorial> javac Main TestInput.txt
error: Class names, 'Main,TestInput.txt', are only accepted if annotation processing is explicitly requested
1 error
PS C:\antlr-projects\ANTLR_Lexer_Tutorial> javac Main.java
Note: Main.java uses or overrides a deprecated API.
Note: Recompile with -Xlint:deprecation for details.
PS C:\antlr-projects\ANTLR_Lexer_Tutorial> java Main TestInput.txt
line 12:13 token recognition error at: '@'
line 16:10 token recognition error at: '#'
[ @0,0:11='// Test NICs',<4>,1:0]
[ @1,14:23='123456789V',<1>,2:0]
[ @2,26:35='123456789X',<1>,3:0]
[ @3,38:49='199012345678',<1>,4:0]
[ @4,54:64='// Test TPs',<4>,6:0]
[ @5,67:76='0123456789',<2>,7:0]
[ @6,81:94='// Test EMAILs',<4>,9:0]
[ @7,97:112='test@example.com',<3>,10:0]
[ @8,115:136='abc@stu.ucsc.cmb.ac.lk',<3>,11:0]
[ @9,139:151='invalid-email',<4>,12:0]
[ @10,153:155='com',<4>,12:14]
[ @11,160:176='// Test ADDRESSES',<4>,14:0]
[ @12,179:202='123 Main Street, Colombo',<4>,15:0]
[ @13,205:214='Apartment ',<4>,16:0]
[ @14,216:226='12/A, Kandy',<4>,16:11]
[ @15,229:256='No.45/1, Galle Rd, Colombo-3',<4>,17:0]
[ @16,259:258='<EOF>',<-1>,18:0]
PS C:\antlr-projects\ANTLR_Lexer_Tutorial> notepad .\ValidationLexer.class

```

```

File Edit Selection View Go Run Terminal Help
Restricted Mode is intended for safe code browsing. Trust this window to enable all features. Manage Learn More

Main.java X
C: > antlr-projects > ANTLR_Lexer_Tutorial > Main.java > ...
1  import org.antlr.v4.runtime.*;
2  import org.antlr.v4.runtime.tree.*;
3
4  import java.io.FileInputStream;
5  import java.io.InputStream;
6
7  public class Main {
8      public static void main(String[] args) throws Exception {
9          String inputFile = args.length>0 ? args[0] : "TestInput.txt";
10         InputStream is = new FileInputStream(inputFile);
11
12         ANTLRInputStream input = new ANTLRInputStream(is);
13         ValidationLexer lexer = new ValidationLexer(input);
14
15         CommonTokenStream tokens = new CommonTokenStream(lexer);
16         tokens.fill();
17
18         for (Token t : tokens.getTokens()) {
19             System.out.println(t.toString());
20         }
21     }
22 }
23

```

Written Question:

1. Regular Expressions and Finite Automata

- ANTLR lexer rules are written using regular expressions or constructs that can be translated into regular expressions.
- Regular languages are sets of strings that can be described using regular expressions. They can be recognized by finite automata.

2. Token Recognition Process

The lexer reads the input as a stream of characters. At each position, ANTLR tries to match the longest possible token using its DFAs. If multiple lexer rules match the same longest string, the rule defined first has priority. Matched tokens are output as a stream of token objects. Each token has Type, Text, Position

3. Role of Lexers in Compilers, Interpreters, and Validators

- Compilers / Interpreters:

Lexers are the first phase of processing. Convert raw source code into tokens that are easier for the parser to understand.

- Validators and Structured Input Processing:

Lexers can be used to validate formats such as NIC numbers, phone numbers, emails, or addresses.

4. Transformation into Executable Lexical Analyzers

1. ANTLR takes your lexer grammar.

2. It generates source files that implement DFAs for all lexer rules.

3. The generated lexer is fully executable and can be integrated into parsers, compilers, or validation programs.